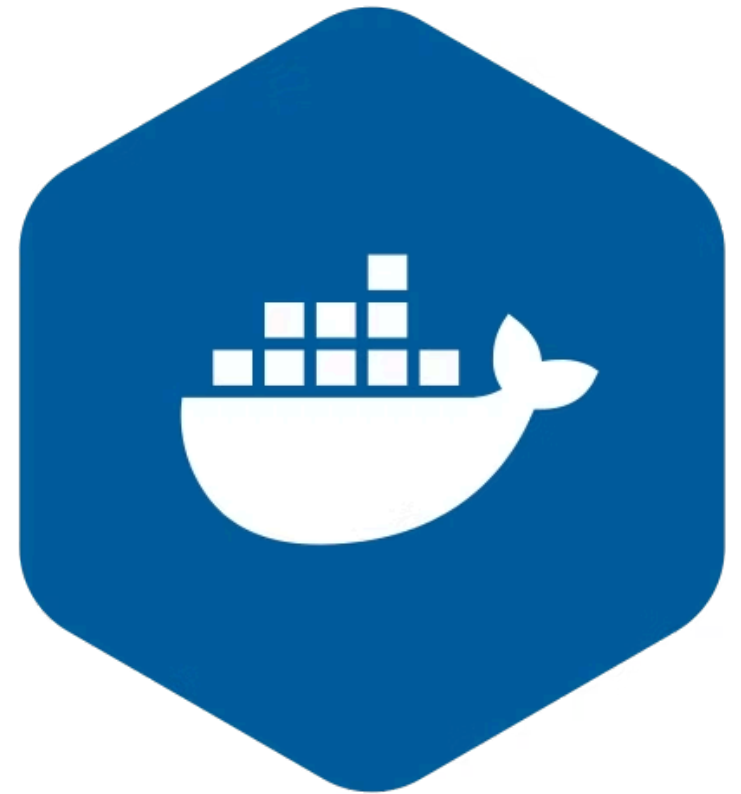


Знакомство с Docker: Упакуй, отправь, запусти

Основы контейнеризации для разработчиков



Что такое Docker и зачем он нужен

Проблема

«На моём компьютере всё работает» — классическая боль каждой команды разработчиков. Разные версии зависимостей, ОС, конфигурации окружения приводят к непредсказуемым сбоям на продакшене.

Решение: Docker

→ Изоляция

Каждое приложение работает в своём окружении

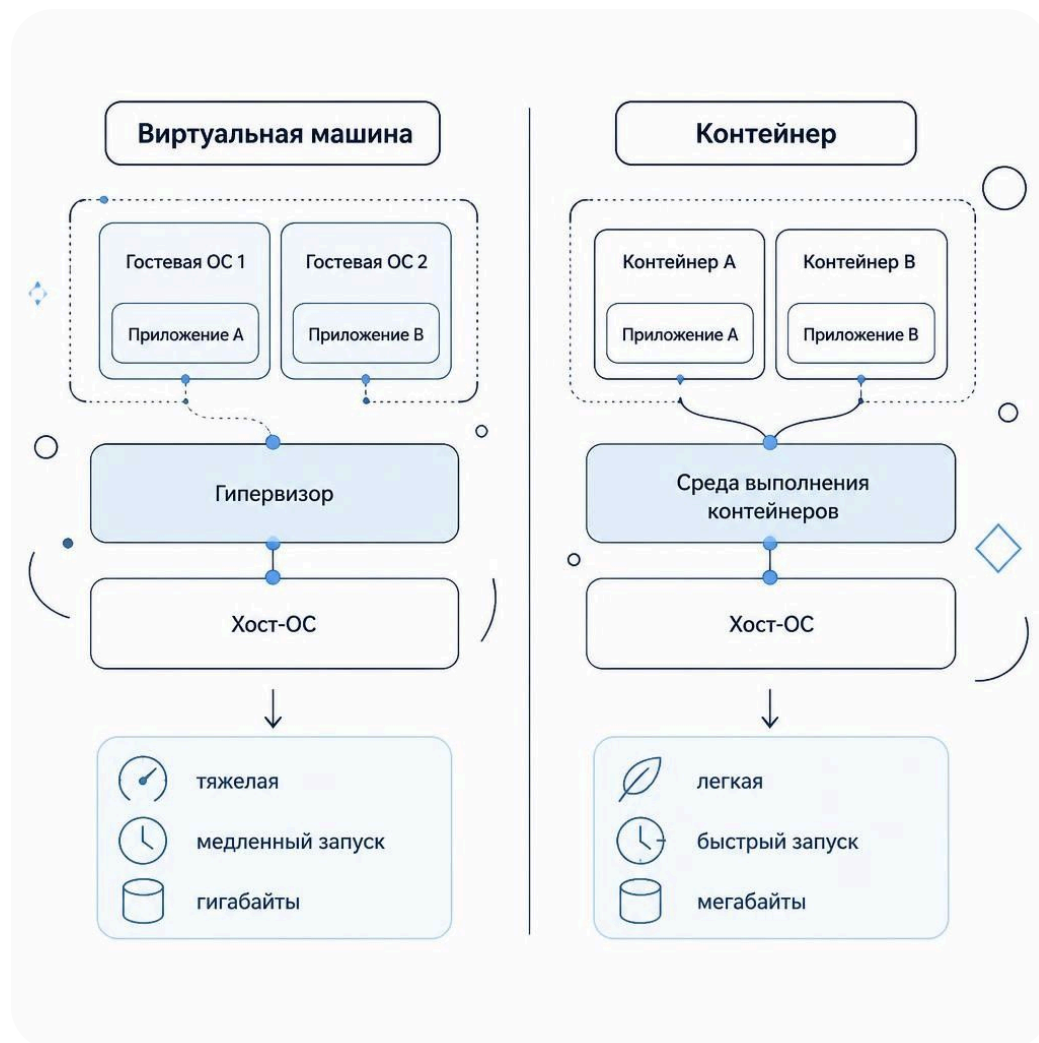
→ Повторяемость

Одинаковое поведение на любой машине

→ Скорость и CI/CD

Быстрый запуск и интеграция в пайплайны

Виртуализация vs Контейнеризация



Ключевые отличия

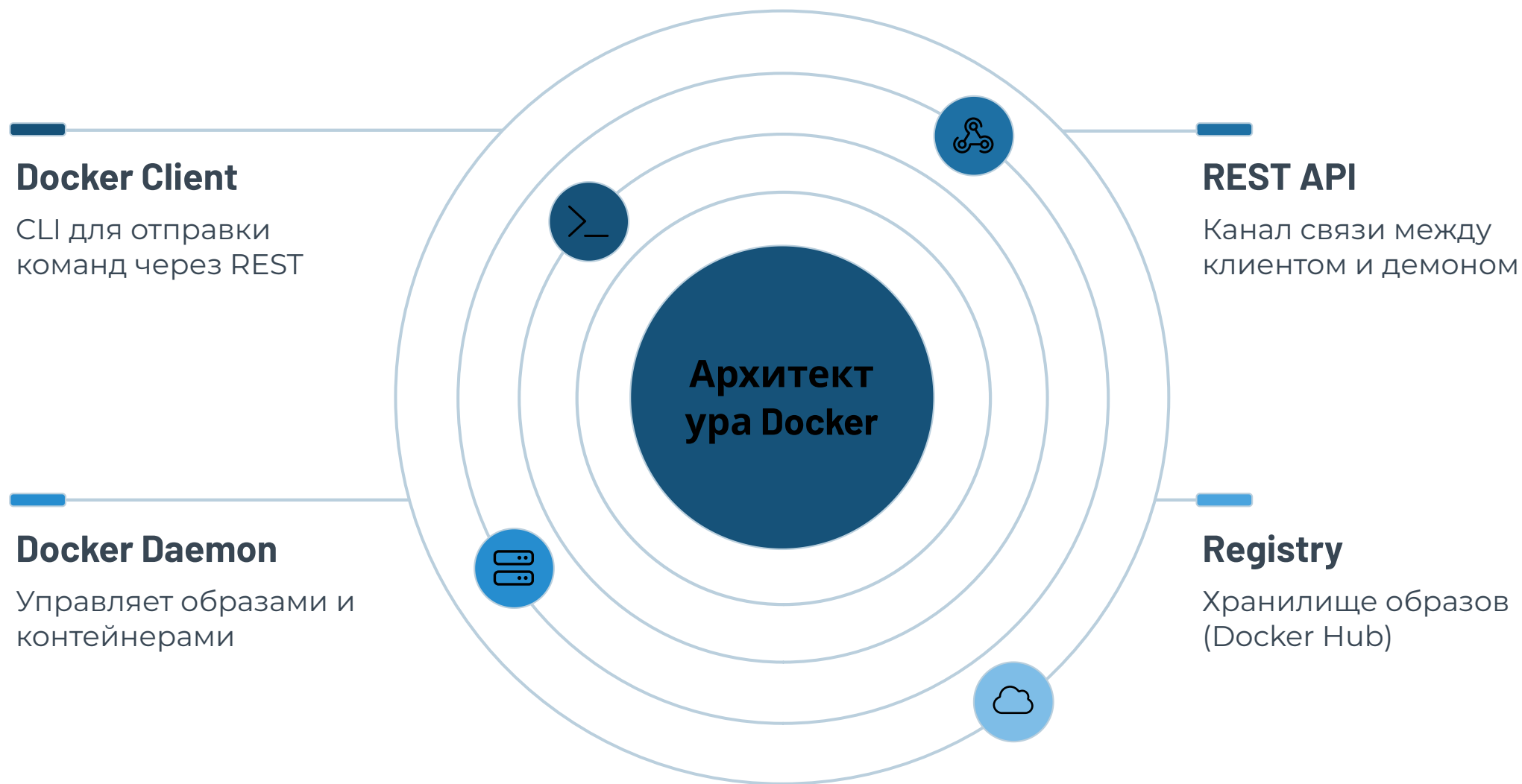
Виртуальная машина

Включает полноценную гостевую ОС. Тяжёлая, запускается минутами, занимает гигабайты.

Контейнер

Использует общее ядро хоста. Лёгкий, стартует за секунды, занимает мегабайты.

Архитектура Docker



Компоненты

- **Docker Client** — CLI-интерфейс для отправки команд
- **Docker Daemon** — фоновый процесс, управляющий контейнерами
- **REST API** — канал связи между клиентом и демоном
- **Registry** — хранилище образов (Docker Hub)

Детали реализации

Docker написан на языке **Go** — это обеспечивает высокую производительность и кроссплатформенность.

Docker Desktop — удобный GUI-инструмент для macOS и Windows, включающий все компоненты в одном пакете.

Основные понятия (Словарь)

Три ключевых концепции, которые нужно понять перед началом работы с Docker:

Dockerfile

Рецепт для создания образа.
Текстовый файл с инструкциями: какую базу взять, что установить, что скопировать и как запустить приложение.

Image / Образ

Аналог **класса** в объектно-ориентированном программировании.
Неизменяемый шаблон, из которого создаются контейнеры.
Хранится в реестре.

Container / Контейнер

Аналог **экземпляра класса**.
Запущенный процесс на основе образа — изолированный, со своей файловой системой и сетью.

 Dockerfile → сборка → Image → запуск → Container. Это фундаментальный цикл Docker.



Образы и реестры (Registry)

Слоистая архитектура образов

Каждый образ состоит из **слоёв (layers)**. Каждая инструкция в Dockerfile создаёт новый слой. Слои кешируются — это ускоряет повторную сборку: если слой не изменился, Docker берёт его из кеша.

Docker Hub

Docker Hub — официальный публичный реестр образов. Здесь хранятся официальные образы: `nginx`, `postgres`, `node`, `python` и тысячи других.

- Загрузка: `docker pull image-name`
- Публикация: `docker push image-name`
- Поиск: `docker search keyword`

Инфраструктура: Сеть, Данные, Логи, Безопасность



Сеть и порты

По умолчанию контейнеры используют **bridge-сеть**. Порты пробрасываются флагом `-p host:container`, например `-p 8080:80`.



Логи

Просмотр вывода контейнера: `docker logs container-name`. Флаг `-f` включает режим реального времени.



Volumes (тома)

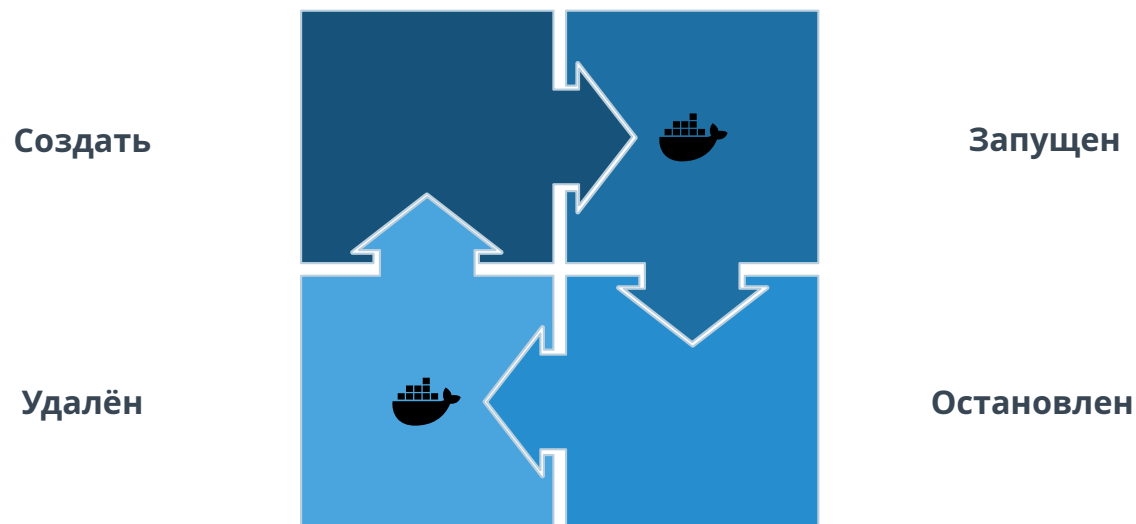
Volumes позволяют сохранять данные между перезапусками контейнера. Без тома данные исчезают при удалении контейнера.



Безопасность

Контейнеры изолированы на уровне процессов ОС с помощью **namespaces** и **cgroups**. Каждый контейнер видит только свои ресурсы.

Жизненный цикл контейнера (Практика)



Основные команды

1

docker run

Создать и запустить контейнер из образа

2

docker ps

Список запущенных контейнеров (-a — все)

3

docker stop

Остановить работающий контейнер

4

docker rm

Удалить остановленный контейнер

Создание своего образа: Dockerfile

Пример простого Dockerfile

```
FROM node:18-alpine

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

CMD ["node", "server.js"]
```

Ключевые инструкции

FROM

Базовый образ — отправная точка сборки

WORKDIR

Рабочая директория внутри контейнера

COPY

Копирование файлов с хоста в образ

RUN

Выполнение команды при сборке образа

CMD

Команда запуска при старте контейнера

Оркестрация: Docker Compose

Пример docker-compose.yml

```
version: '3.8'
services:
  app:
    build: .
    ports:
      - "8080:3000"
    depends_on:
      - db
    environment:
      - DB_HOST=db

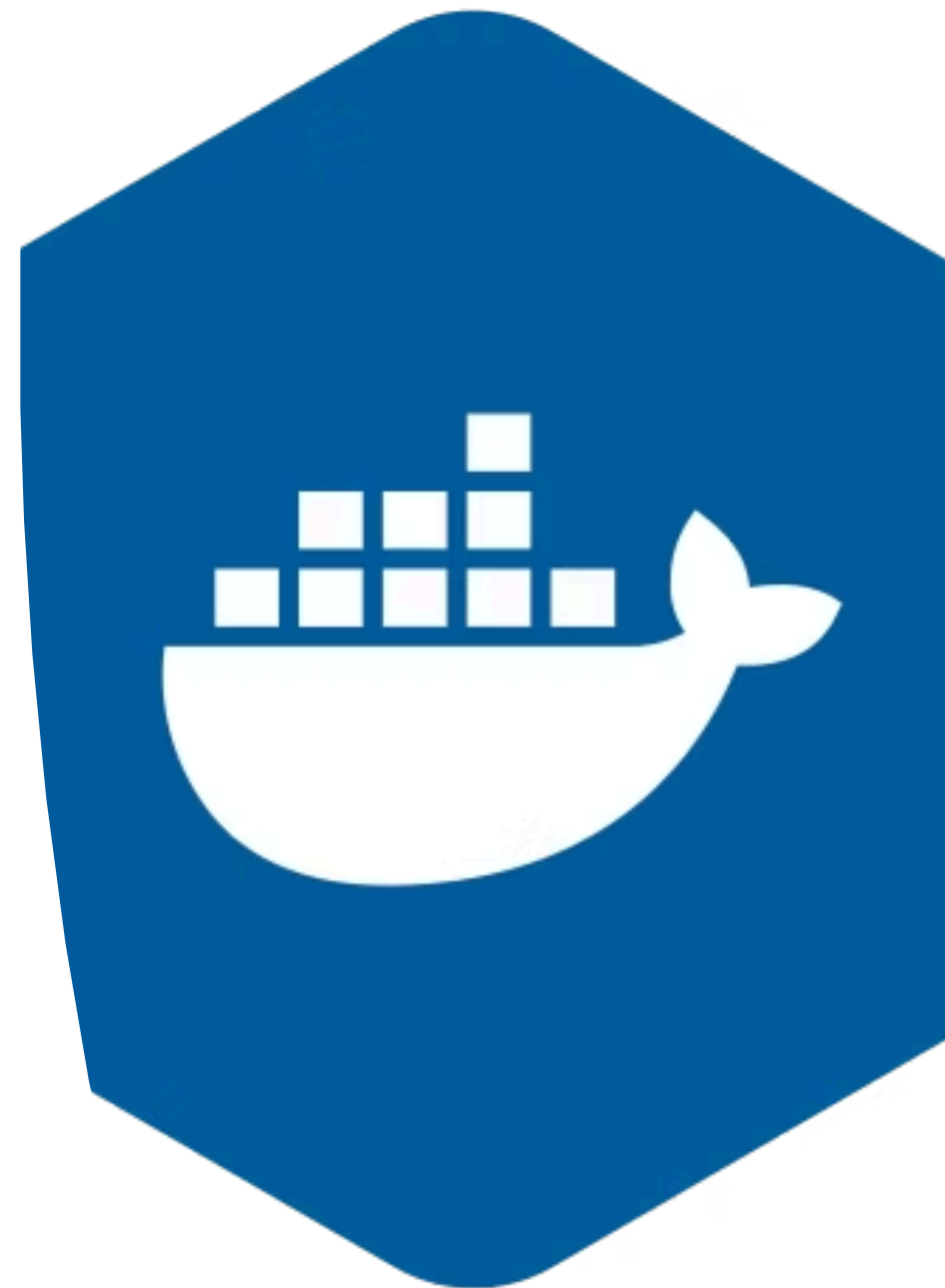
  db:
    image: postgres:15
    volumes:
      -
    pgdata:/var/lib/postgresql/data
    environment:
      -
    POSTGRES_PASSWORD=secret

volumes:
  pgdata:
```

Зачем нужен Compose?

Реальные приложения состоят из нескольких сервисов: веб-сервер, база данных, кеш, очередь сообщений. Docker Compose позволяет описать и запустить их все одной командой.

- ✓ Compose — стандарт для локальной разработки многосервисных приложений.



Запуск

```
docker compose up
```

Остановка

```
docker compose down
```